

Лабораторная работа №7

Студент: Скиба В.А.

Группа: 6204-010302D

Тема: Итераторы, фабричный метод и рефлексия для табулированных функций

1. Цель работы:

- **Цель:** Расширить набор типов табулированных функций добавлением поддержки итерации по точкам (паттерн «Итератор»), реализовать возможность выбирать тип создаваемых табулированных функций динамически (паттерн «Фабричный метод»), а также добавить методы создания через рефлексия. Проверить работоспособность через демонстрационный Main.

2. Краткое описание реализации:

- **Интерфейс:** *Tabulated Function* расширен от *Iterable<Function Point>* — теперь объекты можно использовать в for-each.
- **Итераторы:** В *ArrayTabulatedFunction* и *LinkedList Tabulated Function* добавлены анонимные итераторы:
 - *hasNext()/next()* работают по внутренней структуре (массив / связный список) без вызова публичных методов для доступа к элементам.
 - *next()* возвращает новую копию *FunctionPoint* (чтобы не нарушать инкапсуляцию внутреннего состояния).
 - *remove()* всегда бросает *UnsupportedOperationException*.
 - *next()* бросает *NoSuchElementException*, если элементов нет.
- **Фабрики:** Добавлен интерфейс *TabulatedFunctionFactory* с тремя перегруженными методами *createTabulatedFunction(...)*. Внутри классов реализаций:
 - *ArrayTabulatedFunction.ArrayTabulatedFunctionFactory* — вложенный публичный класс-фабрика.
 - *LinkedListTabulatedFunction.LinkedListTabulatedFunctionFactory* — вложенный публичный класс-фабрика.
- **TabulatedFunctions:** В *TabulatedFunctions*:
 - добавлено приватное статическое поле *factory* (по умолчанию — *ArrayTabulatedFunctionFactory*),
 - метод *setTabulatedFunctionFactory(...)* для смены фабрики,
 - три метода *createTabulatedFunction(...)*, которые делегируют создание текущей фабрике,
 - три дополнительных перегрузки *createTabulatedFunction(Class<? extends TabulatedFunction>, ...)* — создают экземпляры через рефлексия (ищут конструктор с нужной сигнатурой и вызывают его). При ошибке рефлексии бросается *IllegalArgumentException* с оригинальным исключением как причиной.
 - заменены прямые *new ArrayTabulatedFunction(...)* вызовы на *createTabulatedFunction(...)*.

- **Main:** Добавлена демонстрация новых возможностей: итерация `for (FunctionPoint p : tf)`, смена фабрики и проверка рефлексивных методов `createTabulatedFunction(...)` и `tabulate(Class, ...)`.

3. Выполнение заданий (пошагово):

- Задание 1 — Итератор
 - Реализация: *TabulatedFunction* теперь наследует *Iterable<FunctionPoint>*. В классах *ArrayTabulatedFunction* и *LinkedListTabulatedFunction* добавлены анонимные реализации *Iterator<FunctionPoint>*.
 - Проверка: В `Main.demoLab7()` вызван `for (FunctionPoint p : tf)` — он успешно выводит все точки функции (каждая точка — копия *FunctionPoint*, изменения которой не отражаются на исходной функции).

```
@Override
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private int idx = 0;

        @Override
        public boolean hasNext() {
            return idx < pointsCount;
        }

        @Override
        public FunctionPoint next() {
            if (!hasNext()) throw new NoSuchElementException();
            return new FunctionPoint(points[idx++]);
        }

        @Override
        public void remove() {
            throw new UnsupportedOperationException();
        }
    };
}

public static class ArrayTabulatedFunctionFactory implements TabulatedFunctionFactory {
    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new ArrayTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new ArrayTabulatedFunction(leftX, rightX, values);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new ArrayTabulatedFunction(points);
    }
}
```

```

@Override
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private FunctionNode cur = head.next;
        private int idx = 0;

        @Override
        public boolean hasNext() {
            return idx < pointsCount;
        }

        @Override
        public FunctionPoint next() {
            if (!hasNext()) throw new NoSuchElementException();
            FunctionPoint p = new FunctionPoint(cur.point);
            cur = cur.next;
            idx++;
            return p;
        }

        @Override
        public void remove() {
            throw new UnsupportedOperationException();
        }
    };
}

public static class LinkedListTabulatedFunctionFactory implements TabulatedFunctionFactory {
    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new LinkedListTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new LinkedListTabulatedFunction(leftX, rightX, values);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new LinkedListTabulatedFunction(points);
    }
}

```

- Задание 2 — Фабричный метод
 - Реализация: интерфейс *TabulatedFunctionFactory* и вложенные фабрики в двух основных классах. В *TabulatedFunctions* добавлено статическое поле *factory* и методы *createTabulatedFunction(...)*.
 - Проверка: В *Main.demoLab7()*:
 - Сначала табулируем с фабрикой по умолчанию (массивная реализация) и печатаем *tf.getClass()*.
 - Затем ставим *LinkedListTabulatedFunction.LinkedListTabulatedFunctionFactory()* через *TabulatedFunctions.setTabulatedFunctionFactory(...)*, снова табулируем и печатаем класс — видим другой реальный тип.

- Снова ставим `ArrayTabulatedFunction.ArrayTabulatedFunctionFactory()` и убеждаемся, что создаётся массивная реализация.

```
package functions;

public interface TabulatedFunctionFactory {
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount);
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values);
    TabulatedFunction createTabulatedFunction(FunctionPoint[] points);
}
```

- Задание 3 — Рефлексия
 - Реализация: В `TabulatedFunctions` добавлены перегрузки `createTabulatedFunction(Class<? extends TabulatedFunction>, ...)`, которые используют `Class.getConstructor(...)` и `Constructor.newInstance(...)`. Все ошибки рефлексии оборачиваются в `IllegalArgumentException`.
 - Проверка: В `Main.demoLab7()` демонстрация:
 - `createTabulatedFunction(ArrayTabulatedFunction.class, 0, 10, 3)`
 - `createTabulatedFunction(ArrayTabulatedFunction.class, 0, 10, new double[]{0, 10})`
 - `createTabulatedFunction(LinkedListTabulatedFunction.class, new FunctionPoint[]{...})`
 - `tabulate(LinkedListTabulatedFunction.class, new Sin(), 0, Math.PI, 11)`
 - Во всех случаях печатаются классы созданных объектов и их содержимое (через `toString()`), что подтверждает корректность рефлексивного создания.

```

private static TabulatedFunctionFactory factory = new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory();

private TabulatedFunctions() {}

public static void setTabulatedFunctionFactory(TabulatedFunctionFactory f) {
    if (f == null) throw new IllegalArgumentException(s: "Factory cannot be null");
    factory = f;
}

public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
    return factory.createTabulatedFunction(leftX, rightX, pointsCount);
}

public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
    return factory.createTabulatedFunction(leftX, rightX, values);
}

public static TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
    return factory.createTabulatedFunction(points);
}

public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> clazz, double leftX, double rightX, int pointsCount) {
    try {
        Constructor<? extends TabulatedFunction> ctor = clazz.getConstructor(...parameterTypes: double.class, double.class, int.class);
        return ctor.newInstance(leftX, rightX, pointsCount);
    } catch (NoSuchMethodException | InstantiationException | IllegalAccessException | InvocationTargetException e) {
        throw new IllegalArgumentException(e);
    }
}

public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> clazz, double leftX, double rightX, double[] values) {
    try {
        Constructor<? extends TabulatedFunction> ctor = clazz.getConstructor(...parameterTypes: double.class, double.class, double[].class);
        return ctor.newInstance(leftX, rightX, values);
    } catch (NoSuchMethodException | InstantiationException | IllegalAccessException | InvocationTargetException e) {
        throw new IllegalArgumentException(e);
    }
}

public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> clazz, FunctionPoint[] points) {
    try {
        Constructor<? extends TabulatedFunction> ctor = clazz.getConstructor(...parameterTypes: FunctionPoint[].class);
        return ctor.newInstance((Object) points);
    } catch (NoSuchMethodException | InstantiationException | IllegalAccessException | InvocationTargetException e) {
        throw new IllegalArgumentException(e);
    }
}

public static TabulatedFunction tabulate(Class<? extends TabulatedFunction> clazz, Function function, double leftX, double rightX, int pointsCount) {
    if (leftX < function.getLeftDomainBorder() - EPS || rightX > function.getRightDomainBorder() + EPS)
        throw new IllegalArgumentException(s: "Границы табулирования выходят за область определения функции");
    if (pointsCount < 2) throw new IllegalArgumentException(s: "pointsCount < 2");

    double step = (rightX - leftX) / (pointsCount - 1);
    FunctionPoint[] pts = new FunctionPoint[pointsCount];
    for (int i = 0; i < pointsCount; i++) {
        double x = leftX + i * step;
        pts[i] = new FunctionPoint(x, function.getFunctionValue(x));
    }
    return createTabulatedFunction(clazz, pts);
}

```

4. Примеры вывода (фрагменты):

```
private static void demoLab7() {
    System.out.println(x: "\n=== Демонстрация лабораторной работы №7 ===");

    Function f = new functions.basic.Cos();
    TabulatedFunction tf;
    tf = TabulatedFunctions.tabulate(f, leftX: 0, Math.PI, pointsCount: 11);
    System.out.println(tf.getClass());
    System.out.println(x: "Iterating:");
    for (FunctionPoint p : tf) {
        System.out.println(p);
    }

    TabulatedFunctions.SetTabulatedFunctionFactory(new LinkedListTabulatedFunction.LinkedListTabulatedFunctionFactory());
    tf = TabulatedFunctions.tabulate(f, leftX: 0, Math.PI, pointsCount: 11);
    System.out.println(tf.getClass());

    TabulatedFunctions.SetTabulatedFunctionFactory(new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory());
    tf = TabulatedFunctions.tabulate(f, leftX: 0, Math.PI, pointsCount: 11);
    System.out.println(tf.getClass());

    TabulatedFunction t1 = TabulatedFunctions.createTabulatedFunction(clazz: ArrayTabulatedFunction.class, leftX: 0, rightX: 10, pointsCount: 3);
    System.out.println(t1.getClass());
    System.out.println(t1);

    TabulatedFunction t2 = TabulatedFunctions.createTabulatedFunction(clazz: ArrayTabulatedFunction.class, leftX: 0, rightX: 10, new double[] {0, 10});
    System.out.println(t2.getClass());
    System.out.println(t2);

    TabulatedFunction t3 = TabulatedFunctions.createTabulatedFunction(clazz: LinkedListTabulatedFunction.class, new FunctionPoint[] {
        new FunctionPoint(x: 0, y: 0), new FunctionPoint(x: 10, y: 10)
    });
    System.out.println(t3.getClass());
    System.out.println(t3);

    TabulatedFunction t4 = TabulatedFunctions.tabulate(clazz: LinkedListTabulatedFunction.class, new functions.basic.Sin(), leftX: 0, Math.PI, pointsCount: 11);
    System.out.println(t4.getClass());
    System.out.println(t4);
}
```

=== Демонстрация лабораторной работы №7 ===

class functions.ArrayTabulatedFunction

Iterating:

(0.0; 1.0)

(0.3141592653589793; 0.9510565162951535)

(0.6283185307179586; 0.8090169943749475)

(0.9424777960769379; 0.5877852522924731)

(1.2566370614359172; 0.30901699437494745)

(1.5707963267948966; 6.123233995736766E-17)

(1.8849555921538759; -0.30901699437494734)

(2.199114857512855; -0.587785252292473)

(2.5132741228718345; -0.8090169943749473)

(2.827433388230814; -0.9510565162951535)

(3.141592653589793; -1.0)

class functions.LinkedListTabulatedFunction

class functions.ArrayTabulatedFunction

class functions.ArrayTabulatedFunction

{{(0.0; 0.0), (5.0; 0.0), (10.0; 0.0)}}

class functions.ArrayTabulatedFunction

{{(0.0; 0.0), (10.0; 10.0)}}

class functions.LinkedListTabulatedFunction

{{(0.0; 0.0), (10.0; 10.0)}}

class functions.LinkedListTabulatedFunction

{{(0.0; 0.0), (0.3141592653589793; 0.3090169943749474), (0.6283185307179586;

0.5877852522924731), (0.9424777960769379; 0.8090169943749475),

(1.2566370614359172; 0.9510565162951535), (1.5707963267948966; 1.0),

(1.8849555921538759; 0.9510565162951536), (2.199114857512855; 0.8090169943749475),

(2.5132741228718345; 0.5877852522924732), (2.827433388230814; 0.3090169943749475),

(3.141592653589793; 1.2246467991473532E-16)}}

5. Краткие выводы:

- Все три задания выполнены: итераторы позволяют использовать for-each, фабрика и её смена дают возможность выбирать конкретную реализацию, а рефлексия позволяет создавать объекты на основании класса в рантайме. В демонстрации Main показаны примеры использования всех новых возможностей.